



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-40376 Visual Studio Code Elevation of Privilege Vulnerability

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-11
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-40376 Visual Studio Code Elevation of Privilege Vulnerability - Vulnerability Report

Published: 2026-06-11 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

CVE-2026-40376 is a high-severity Microsoft Visual Studio Code elevation-of-privilege vulnerability disclosed in the June 2026 Microsoft security release. Microsoft, the CVE Program CNA record, and NVD align on the public description: improper input validation in Visual Studio Code can allow an unauthorized attacker to elevate privileges over a network. The authoritative Microsoft CVRF record assigns CVSS v3.1 `7.5 / HIGH` with vector `AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:H`, marks the issue as `Important`, and states the reviewed exploitability assessment was `Exploitation Less Likely` at release.

The public technical record remains sparse, but the Microsoft CVRF FAQ adds an important clue: successful exploitation may let an attacker obtain permissions associated with an MCP Server managed identity. That wording materially changes the defensive interpretation. Instead of treating the flaw as a generic local editor bug, defenders should evaluate whether Visual Studio Code instances are integrated with remote MCP-backed workflows, Entra-protected services, or cloud resources reachable through a managed identity trust path. Microsoft does not publish a full exploit narrative in the reviewed public sources, so this MCP / managed-identity angle should be treated as high-value vendor context with only medium confidence on exact trigger conditions.

Operational risk is highest where developers use Visual Studio Code to interact with remote services, AI-agent tooling, or internal cloud-connected workflows, and where the resulting identity context can reach sensitive data or control-plane functions. Because user interaction is required and attack complexity is high, this does not currently present like a mass internet worm. It is better modeled as a targeted social-engineering or workflow-abuse opportunity against developer workstations and engineering users who can be induced to open content, connect to infrastructure, or approve a remote interaction path that Visual Studio Code should have constrained more safely.

Immediate priorities are to upgrade Visual Studio Code to `1.123.2` or later, identify business workflows that rely on MCP servers or comparable remote tool integrations, review allowlists and identity scope for those services, and hunt for anomalous VS Code network connections or post-connection cloud activity that could indicate abuse of the vulnerable path.

Executive takeaway	Assessment
Business impact	A successful exploit can expose the permissions of a connected identity context rather than only editor-local state, creating potential access to code, secrets, or cloud resources reachable through the MCP-associated trust chain.
Attacker requirement	No prior privileges are required, but exploitation is remote, user interaction is required, and Microsoft rates attack complexity as high.
Public exploitation status	No reviewed authoritative source confirmed public disclosure or in-the-wild exploitation as of 2026-06-11.
Immediate priority	Upgrade to `1.123.2+`, review remote MCP and identity-backed integrations, and investigate suspicious VS Code-originated network connections and downstream cloud activity.

2. Intelligence Requirement

This review addresses the following intelligence requirement: what does the current public record say about `CVE-2026-40376`, which Visual Studio Code deployments and workflows are most exposed, how credible is near-term exploitation risk, and what should defenders do immediately to reduce the chance that VS Code-mediated remote interactions inherit or expose sensitive managed-identity permissions?

Question	Assessment
What is the vulnerability?	An improper-input-validation flaw in Visual Studio Code that can allow an unauthorized attacker to elevate privileges over a

	network.
Which versions are affected?	The CNA record marks Visual Studio Code versions `1.0.0` through `< 1.123.2` as affected.
What access does the attacker need?	None, but the user must interact with attacker-controlled or attacker-influenced content or workflow material.
Why does the network attack vector matter?	It suggests the exploit path involves remote content, remote service interaction, or a network-facing trust boundary rather than a purely local memory-corruption or filesystem-only bug.
What is the most important vendor clue?	Microsoft's CVRF FAQ says successful exploitation could obtain permissions associated with an MCP Server managed identity.
What matters most to defenders?	Patch quickly, reduce unnecessary identity scope on MCP-connected services, and monitor for suspicious VS Code network activity followed by cloud-resource access or policy-relevant authentication events.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Update Guide entry	Canonical advisory title, current status page, and remediation linkage.	High
Microsoft CVRF OData update mapping	Ties the CVE to the June 2026 security release.	High
Microsoft June 2026 CVRF document	Authoritative CVSS vector, exploitability assessment, affected product identity, fixed build `1.123.2`, acknowledgment, and the MCP managed-identity FAQ language.	High
CVE Program CNA API record	Confirms affected-version range, weakness mapping, and CVSS v3.1 metadata.	High
NVD JSON 2.0 record	Corroborates the public description and normalized CVSS details in a broadly consumed public feed.	High
Visual Studio Code June 2026 release notes	Confirms `1.123.2` as the maintenance build available to users.	Medium
CISA KEV JSON feed snapshot	Confirms the CVE was not listed in KEV at review time.	High

Confidence is high on the existence of the vulnerability, the affected-version range, the fixed build, the severity, and the absence of reviewed public exploitation evidence because those facts are directly supported by Microsoft, the CNA record, NVD, and CISA. Confidence is only medium on the precise exploit chain because Microsoft's public description is terse and the reviewed public source set does not expose a proof of concept, root-cause patch narrative, or a vendor-confirmed reproduction sequence.

The most analytically important ambiguity is the Microsoft CVRF FAQ statement about obtaining permissions associated with an MCP Server managed identity. That is vendor-origin information and therefore meaningful, but the advisory does not publish enough implementation detail to determine whether exploitation requires a crafted workspace, a remote MCP

connection flow, unsafe approval semantics, or another network-mediated trust edge inside Visual Studio Code. Detection and hunting guidance in this report is therefore intentionally behavior-based and focused on exposed workflows and suspicious post-connection outcomes rather than on a brittle exploit signature.

4.1 Core Vulnerability Metadata

Field	Value
CVE	`CVE-2026-40376`
Vulnerability name	`Visual Studio Code Elevation of Privilege Vulnerability`
Weakness	`CWE-20` Improper Input Validation
Severity	`7.5 / HIGH` (CVSS v3.1)
CVSS v3.1 vector	`AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:H`
Microsoft temporal vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:H/E:U/RL:O/RC:C`
Attack vector	Network
Attack complexity	High
Privileges required	None
User interaction	Required
Scope	Unchanged
Impact profile	High confidentiality, integrity, and availability impact in the reviewed public record
Microsoft severity label	Important
Product	Visual Studio Code
Affected versions	`>= 1.0.0` and `< 1.123.2`
Fixed version	`1.123.2`
Public disclosure at release	No
Exploitation at release	No
Microsoft exploitability assessment	Exploitation Less Likely
Customer action required	Yes
Acknowledged finder	Adrian Frei

4.2 Exposure Interpretation

Evidence point	What it suggests	Confidence
CNA and NVD describe a network-reachable elevation-of-privilege condition with improper input validation.	The vulnerable boundary likely involves processing remote content, remote authority, or a service-mediated workflow rather than an offline local-only parser	High

	flaw.	
Microsoft's CVRF FAQ says successful exploitation could obtain permissions associated with an MCP Server managed identity.	The practical blast radius may extend into cloud or service permissions reachable through VS Code-connected MCP infrastructure.	Medium
User interaction is required and attack complexity is high.	Exploitation is more likely to be delivered through targeted social engineering, malicious project content, or workflow abuse than through indiscriminate scanning.	High
Fixed build is `1.123.2`.	Organizations cannot assume earlier `1.123.x` builds are safe; explicit version verification matters.	High
No reviewed source published exploit code, stable IOCs, or campaign attribution.	Defenders must favor behavior-based detections, version-state inventory, and identity-scope review over exploit-signature matching.	High

4.3 Highest-Risk Exposure Patterns

Exposure pattern	Why it matters
Developer workstations using VS Code with remote MCP-backed tooling	A compromise path that inherits MCP-associated permissions can expose sensitive development, data-access, or control-plane workflows.
VS Code instances connected to cloud resources through managed identities or delegated service principals	Even if the editor itself is not privileged, the identity context behind a connected tool may be.
Users who frequently open third-party repositories, shared workspaces, or externally supplied project files	User interaction is required, making social engineering and content-lure delivery the most credible initial vector.
High-value engineers, build operators, and cloud administrators using VS Code as a daily control surface	Their downstream permissions make lower-volume, higher-complexity exploitation more operationally attractive.

4.4 Analyst Findings

Finding	Defensive implication
The best vendor clue ties successful exploitation to MCP Server managed-identity permissions rather than to generic editor-only impact.	Patch prioritization should consider identity reach and remote tooling integrations, not just local endpoint criticality.
The affected-version range spans essentially the full historical Visual Studio Code install base below `1.123.2`.	Exposure can be broad wherever self-managed or slowly updated developer endpoints exist.
The attack requires user interaction and high complexity.	Security awareness, content provenance, and approval-surface hardening remain relevant mitigations alongside patching.
Public detail is too limited to support a single exploit narrative with high confidence.	Hunting should focus on suspicious VS Code network behavior, unusual MCP usage, and cloud-resource activity rather than on one presumed artifact.

5. Threat Context

CVE-2026-40376 is best understood as a trust-boundary problem in a modern developer workflow rather than a classic local privilege-escalation bug. The combination of `AV:N`, `PR:N`, `UI:R`, and `AC:H` implies an attacker can influence a victim over a network path but still needs the victim to take an action that places Visual Studio Code into the vulnerable state. In practical terms, that points to adversaries using malicious repositories, crafted project artifacts, phishing lures, or workflow-specific prompts to induce a developer to open, connect, or approve something that should have been validated more safely.

The most meaningful public Microsoft clue is the FAQ statement that successful exploitation could obtain permissions associated with an MCP Server's managed identity. If that statement reflects the operative trust boundary, then the real impact is not merely that VS Code misbehaves locally; it is that the editor can become the bridge by which an attacker reaches identity-backed cloud resources, enterprise APIs, internal knowledge stores, or automation surfaces exposed through MCP-connected tooling. This makes the issue especially relevant to organizations that have rapidly adopted AI-assisted development and MCP-integrated internal services without tightly minimizing identity scope.

At the same time, the reviewed source set does not justify overstatement. No reviewed authoritative source tied the CVE to named threat actors, public proof-of-concept release, known exploitation in the wild, or CISA KEV inclusion as of 2026-06-11. Defenders should therefore calibrate messaging carefully: the vulnerability appears serious and worth prompt remediation because of where it lives in the workflow and what identity context may be reachable, but current public evidence does not support claims of active broad exploitation.

ATT&CK mapping	Relevance	Confidence
`T1204.001` User Execution: Malicious File	Plausible delivery pattern if exploitation requires opening a crafted workspace, project artifact, or other attacker-supplied content in VS Code.	Medium
`T1071.001` Application Layer Protocol: Web Protocols	Plausible for the network-mediated interaction path between VS Code and an attacker-controlled or attacker-influenced service endpoint.	Medium
`T1078.004` Valid Accounts: Cloud Accounts	Plausible downstream effect if the attacker can act with permissions associated with a managed identity or equivalent cloud-backed trust context.	Medium
Additional ATT&CK techniques	Not assigned with high confidence because the reviewed public record does not expose a complete intrusion chain.	High

6. Detection and Hunting

Because the reviewed public sources do not expose exploit payloads, packet fingerprints, or stable IOCs, the most defensible monitoring strategy is layered: identify unpatched VS Code installations, map which users and endpoints rely on MCP or similar remote-tool integrations, detect anomalous VS Code network connections to unapproved endpoints, and correlate those events with downstream cloud-resource access or authentication activity linked to the same user, host, or managed identity.

6.1 Detection Logic Summary

Signal	Telemetry source	Analytic goal
Endpoints running Visual Studio Code ` <code><1.123.2`</code>	Software inventory, EDR device inventory, package-management telemetry	Identify exposed systems and prioritize upgrades
VS Code making outbound network connections to non-allowlisted hosts,	EDR network telemetry, proxy logs, DNS	Surface suspicious remote interaction paths

especially immediately after opening a workspace or remote project	logs, firewall logs	that may trigger the vulnerable condition
Presence or modification of `.vscode/mcp.json`, `.code-workspace`, and high-risk `.vscode/settings.json` content on endpoints that later show suspicious network behavior	EDR file telemetry, FIM, source-control audit context	Identify developer content that may have introduced unsafe connection or tool configuration
Authentication or cloud-resource activity shortly after suspicious VS Code remote interaction	Entra sign-in logs, cloud audit logs, service-specific access logs	Detect possible abuse of inherited identity context after the initial trigger
VS Code processes spawning helper processes or invoking MCP-related local commands unexpectedly	EDR process telemetry, Sysmon Event ID 1, Windows 4688, macOS/Linux audit logs	Add local context to remote interaction and triage whether a malicious workflow was staged

Analyst caveat: many legitimate development workflows now include MCP server connections, remote repositories, and cloud-backed interactions. Triage should therefore focus on novelty, unapproved domains, unusual timing, unusual identity scope, and whether the endpoint was still below `1.123.2` at the time of activity.

6.2 Example Sigma Logic

```

title: Suspicious Visual Studio Code Connection to Unapproved Remote Service
id: cve-2026-40376-vscode-remote-connection
status: experimental
description: Detects Visual Studio Code connecting to non-allowlisted remote hosts, which may warrant review during response to CVE-2026-40376.
logsource:
  category: network_connection
  product: windows
detection:
  selection_process:
    Image|endswith:
      - '\\Code.exe'
      - '\\code.exe'
      - '\\Code - Insiders.exe'
      - '\\code-insiders.exe'
  filter_known_private_ranges:
    DestinationIp|cidr:
      - '10.0.0.0/8'
      - '172.16.0.0/12'
      - '192.168.0.0/16'
      - '127.0.0.0/8'
      - '169.254.0.0/16'
  filter_known_domains:
    DestinationHostname|endswith:
      - '.corp.example'
      - '.trusted-mcp.example'
      - '.microsoft.com'
      - '.github.com'
  condition: selection_process and not 1 of filter_known_*
falsepositives:
  - legitimate developer use of approved third-party MCP servers or remote services not yet represented in the allowlist
  - remote development workflows using ephemeral lab or cloud hostnames
level: medium

```

6.3 Example Hunt Queries

Microsoft Defender XDR / Sentinel-style hunt for VS Code connections to non-private, non-allowlisted destinations:

```

DeviceNetworkEvents
| where Timestamp > ago(30d)

```



```

| where InitiatingProcessFileName in~ ("Code.exe", "code.exe", "Code - Insiders.exe", "code-
insiders.exe", "code", "code-insiders")
| where not(ipv4_is_private(RemoteIP))
| where RemoteUrl !endswith ".microsoft.com"
| where RemoteUrl !endswith ".github.com"
| where RemoteUrl !endswith ".visualstudio.com"
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp), ConnectionCount=count(),
Ports=make_set(RemotePort, 20) by DeviceName, AccountName, RemoteUrl, RemoteIP,
InitiatingProcessCommandLine
| order by LastSeen desc

```

Hunt for newly modified workspace or MCP configuration followed by suspicious VS Code egress from the same endpoint:

```

let suspiciousFiles = DeviceFileEvents
| where Timestamp > ago(30d)
| where FileName in~ ("mcp.json", "settings.json") or FileName endswith ".code-workspace"
| where FolderPath has_any ("\\.\vscode\\", "\\.vscode/")
| project DeviceId, DeviceName, FileModTime=Timestamp, FolderPath, FileName, SHA1,
InitiatingProcessFileName, InitiatingProcessCommandLine;
let suspiciousNetwork = DeviceNetworkEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("Code.exe", "code.exe", "Code - Insiders.exe", "code-
insiders.exe", "code", "code-insiders")
| where not(ipv4_is_private(RemoteIP))
| project DeviceId, NetTime=Timestamp, DeviceName, AccountName, RemoteUrl, RemoteIP, RemotePort,
InitiatingProcessCommandLine;
suspiciousFiles
| join kind=inner suspiciousNetwork on DeviceId
| where NetTime between (FileModTime .. FileModTime + 2h)
| project FileModTime, NetTime, DeviceName, AccountName, FolderPath, FileName, RemoteUrl, RemoteIP,
RemotePort, InitiatingProcessFileName, InitiatingProcessCommandLine
| order by NetTime desc

```

Cloud-side follow-up hunt pattern: look for managed-identity or service-principal activity from resources associated with developer tooling shortly after suspicious VS Code sessions. Exact queries will vary by platform, but the key analytic question is whether a developer endpoint's suspicious remote interaction is followed by unusual token issuance, API calls, or resource access under an identity path that normally remains low-volume or tool-scoped.

6.4 Hunting Guidance

Hunt question	Why it matters
Which endpoints were still running VS Code below `1.123.2` after 2026-06-09?	These systems form the highest-priority review and patch set.
Which of those endpoints use MCP or other remote service integrations from inside VS Code?	The vendor FAQ makes identity-backed remote workflows the most operationally relevant exposure area.
Did suspicious `.vscode/mcp.json`, `.code-workspace`, or related project files appear shortly before unusual VS Code network activity?	That sequence is consistent with content-mediated exploitation or workflow staging.
Did any suspicious VS Code session precede unusual cloud-resource access, token issuance, or policy alerts tied to a managed identity or delegated service principal?	That is the most important downstream impact path suggested by Microsoft.
Were unusual VS Code remote connections dismissed as normal developer activity without checking version state and allowlists?	Legitimate remote tooling can camouflage abuse unless defenders correlate destination, timing, and identity context.

7. Recommendations

Priority	Owner	Action
Critical	Endpoint Engineering / Developer Platform	Upgrade Visual Studio Code to `1.123.2` or

		later across all managed developer endpoints, including user-installed and insider variants.
Critical	Vulnerability Management	Verify remediation by installed version inventory rather than by assumption that a general June patch cycle covered the editor.
High	Cloud Security / Platform Engineering	Review MCP-connected services and reduce managed-identity permissions to the minimum required for each workflow; remove broad contributor, data-reader, or secrets-access grants that are not essential.
High	Security Architecture	Establish an allowlist for approved MCP or remote service destinations used by VS Code and alert on exceptions.
High	SOC / Detection Engineering	Deploy hunts for suspicious VS Code outbound network connections, recent <code>.vscode/mcp.json</code> or <code>.code-workspace</code> changes, and follow-on identity or cloud-resource activity.
High	Engineering Leadership	Reinforce guidance that developers should avoid opening externally sourced workspaces or enabling unfamiliar remote-tool integrations until the endpoint is confirmed patched.
Medium	IAM / Entra Administration	Review Conditional Access, token-protection, and audit coverage for services used through VS Code-integrated workflows so suspicious token use is visible and attributable.
Medium	Threat Intelligence	Re-check Microsoft advisories, release notes, KEV status, and public research in subsequent patch cycles for stronger technical disclosure or evidence of exploitation.

Additional analyst guidance:

- Treat this as a workflow and identity-exposure problem, not just an editor-version problem. A lightly used endpoint with access to a powerful managed identity can matter more than a large fleet of lower-value laptops.
- If immediate upgrade is not possible, document the exception, restrict or temporarily disable high-risk MCP integrations where feasible, and add temporary monitoring around VS Code outbound traffic and cloud-resource access.
- Do not overfit detections to one hostname or one project artifact. The public record supports category-based monitoring: untrusted project content, unusual remote connections, and identity-backed downstream activity.

8. References

- 1. Microsoft Security Update Guide, CVE-2026-40376:
<https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-40376>
- 2. Microsoft CVRF OData update mapping for CVE-2026-40376:
<https://api.msrc.microsoft.com/cvrf/v3.0/updates/CVE-2026-40376>
- 3. Microsoft June 2026 CVRF document: <https://api.msrc.microsoft.com/cvrf/v3.0/cvrf/2026-Jun>

- 4. CVE Program CNA API record: <https://cveawg.mitre.org/api/cve/CVE-2026-40376>
- 5. NVD JSON 2.0 API query: <https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-40376>
- 6. Visual Studio Code June 2026 release notes (`1.123` / update `1.123.2`): https://code.visualstudio.com/updates/v1_123
- 7. CISA Known Exploited Vulnerabilities JSON feed:
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json